ENGINEERING WITH AI CONTRIBUTORS

QUICK REFERENCE FOR BOUNDED, VERIFIABLE AI CONTRIBUTIONS

Use this reference when asking AI to analyze, design, implement, test, review, document, release, or operate software. The objective is not a longer prompt. The objective is a contribution another engineer can understand, verify, and accept or reject.

Before opening the AI tool

1. What single engineering outcome is needed?	**2. Which artifacts and people have authority?**	**3. What may the AI inspect, modify, or execute?**
4. What evidence would support acceptance?	**5. Which condition requires the AI to stop?**	**If unclear, do not begin implementation.**

Construct the contribution contract

ROLE Name one engineering responsibility. Separate planning, implementation, review, and release authority.	**OBJECTIVE** State one observable outcome. Describe the result, not the desired amount of output.
AUTHORITY Name governing requirements, ADRs, contracts, policies, source files, and accountable owners.	**CURRENT STATE** State what exists, what was verified, what is broken, and what remains unknown. Inspect before editing.
SCOPE Define what the AI may inspect, create, modify, and execute.	**EXCLUSIONS** Protect files, contracts, schemas, dependencies, services, environments, and decisions.
CONSTRAINTS Preserve architecture, compatibility, security, privacy, state, accessibility, and operational invariants.	**DELIVERABLES** Name required code, tests, diagrams, documentation, findings, and evidence reports.
ACCEPTANCE Define observable behavior and failure conditions. Completion is not acceptance.	**VERIFICATION** Specify tests, Docker or database checks, startup, browser journeys, accessibility, and security evidence.
REPORTING Require files, rationale, commands, passed, failed, and not-run checks, limitations, and returned decisions.	**STOP CONDITIONS** Stop for missing authority, conflicting sources, failed baseline, protected data, expanded scope, or unresolved decisions.

```
ROLE -> OBJECTIVE -> AUTHORITY -> CURRENT STATE -> SCOPE -> EXCLUSIONS -> CONSTRAINTS -> DELIVERABLES -> ACCEPTANCE -> VERIFICATION -> REPORTING -> STOP
```

OPERATE AND JUDGE THE CONTRIBUTION

AI CONTRIBUTES. PEOPLE REMAIN ACCOUNTABLE.

ENGINEERING NEED	AI CONTRIBUTION ROLE	HUMAN DECISION
Clarify a problem	Problem-framing partner	Accept intent and priority
Recover project knowledge	Context reviewer	Resolve source authority
Compare designs or record a decision	Architecture decision partner	Accept trade-offs and boundaries
Decompose approved work	Planner	Approve readiness and sequence
Generate bounded source changes	Implementation engineer	Own and accept changed software
Design independent evidence	Test and evidence engineer	Accept oracle and sufficiency
Challenge a change	Code, architecture, or security reviewer	Correct, split, reject, or accept risk
Evaluate an interface	UX or accessibility reviewer	Accept product experience
Prepare release or incident evidence	Release or incident-analysis partner	Authorize action and residual risk

VERIFY BEFORE ACCEPTANCE

1. Inspect every changed artifact.
2. Run focused tests and the approved regression suite.
3. Start the real application.
4. Use Docker or representative dependencies when environment matters.
5. Perform the browser, API, or operator journey.
6. Exercise failure and recovery behavior.
7. Separate passed, failed, and not-run checks.
8. Confirm scope and record limitations.

The AI reports what it ran and what the human must still run. Missing access is not passing evidence.

FACTUAL COMPLETION REPORT

1. Behavior implemented or analyzed.
2. Artifacts changed and rationale.
3. Commands and tools executed.
4. Passed, failed, and not-run checks.
5. Functional checks assigned to the human.
6. Limitations and residual risks.
7. Decisions returned to accountable owners.

Completion means the actual state is understandable. It does not mean the contribution is accepted.

DECIDE

PASS	Evidence supports the bounded claim. Limitations are explicit and acceptable.
REVISE	The direction is usable, but a missing decision, failed check, or bounded correction prevents acceptance.
STOP	Continuing would invent intent, cross authority, hide failure, expose protected data, or create unauthorized consequence.

HUMAN AUTHORITY REMAINS	Product intent and priority | consequential architecture | security and privacy risk | acceptance and residual risk | release authority | production action and incident command | ownership of accepted software

```
ACCEPTED INTENT -> AUTHORITATIVE CONTEXT -> BOUNDED CONTRIBUTION -> INDEPENDENT EVIDENCE -> ACCOUNTABLE ACCEPTANCE -> RETAINED LEARNING
```

Design before generation. Evidence before acceptance. Human ownership throughout the lifecycle.